How Indexes Accelerate Queries

From full scans to covering indexes

SKILL Application Test

A query, its data structure, and the work it performs

Start with one query and a fixed result

```
SELECT created_seq, amount_cents
FROM orders
WHERE tenant_id = 42
ORDER BY created_seq DESC
LIMIT 20;
```

200,000 synthetic orders

100 tenants; 1,980 matching rows
created_seq = creation order
amount_cents = amount in cents
payload = 64 bytes per row

Measure the work

Count SQLite virtual-machine (VM) instructions. Time full result retrieval: 3 warm-ups, then 11 executions. Report the median with a warm cache.

Keep the result fixed

Verify identical rows and order after every index change.
The access path is the experimental variable; the requested result stays fixed.

An index stores an additional order over the data

Row-ID order

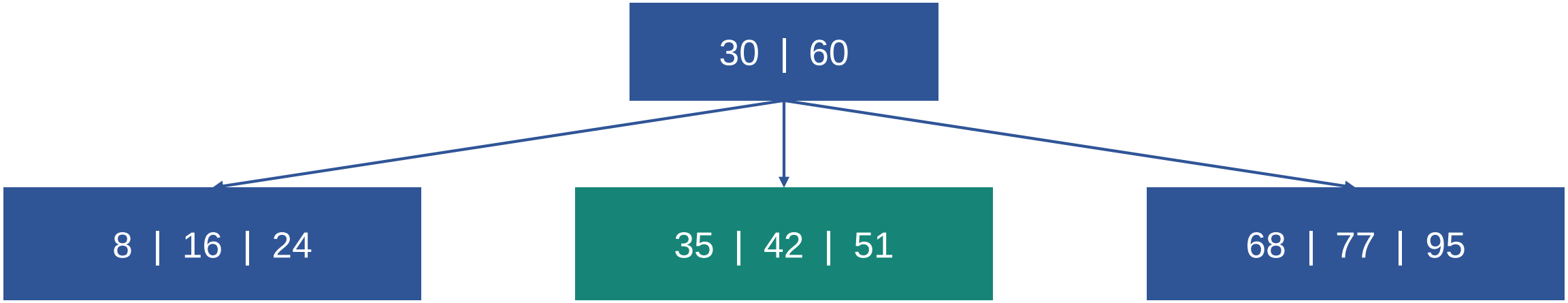
Row ID	Tenant	Sequence	Amount
17	42	16	8100
28	7	27	6400
66	42	65	2200
91	12	90	3500

Tenant-key order

Tenant	Row ID
7	28
12	91
42	17
42	66

These illustrative keys place tenant 42’s row IDs 17 and 66 next to each other.
The row IDs locate the corresponding records in the original table.

A B-tree routes the search through a few pages



Search key 42

Compare separator keys, descend into the middle range, and find the matching entry.

30 < 42 < 60: follow the middle child, then locate 42 among 35, 42 and 51.

Illustrative keys; each node represents a storage page.

High fan-out keeps the tree shallow

Index page type	Pages	Entries	Storage
Internal	6	845	24 KiB
Leaf	846	199,155	3.305 MiB

Fan-out is the number of child links in an internal page.

A three-level structure

The measured index has a root,
an internal level and leaf pages.
Its longest path contains 2 edges.

Work follows the access path

A lookup routes to a small region.
A full scan visits the index or table
sequentially across its entries.

Without an index, filter and sort the table



EXPLAIN QUERY PLAN reports:

```
SCAN orders
USE TEMP B-TREE FOR ORDER BY
```

Baseline: 617,851 virtual-machine instructions to retrieve the 20 rows.

A tenant index removes most filtering work

```
CREATE INDEX idx ON orders(tenant_id);
```



The equality constraint is searchable

SEARCH orders USING INDEX idx (tenant_id=?)

The requested order remains separate

A temporary B-tree still supplies ordering. Measured work falls to 23,792 instructions.

A composite index groups and orders together

```
CREATE INDEX idx ON orders(tenant_id, created_seq);
```

Tenant	Created sequence	Illustrative key order
41	199800	Previous tenant
42	100	Start of tenant 42
42	95000	Same tenant, increasing sequence
42	199943	Recent row inside the same range
43	15	Next tenant

Equality on tenant selects one contiguous range; sequence orders the entries within it.

An ordered index can stop after the first 20 matches




```
SEARCH orders USING INDEX idx (tenant_id=?)
```

The order comes from the index

The plan no longer reports
USE TEMP B-TREE FOR ORDER BY.

Early termination changes the workload

Measured instructions: 150.
The query retrieves a small tail of
one tenant’s ordered range.

A covering index supplies every requested value

```
CREATE INDEX idx ON orders(tenant_id, created_seq, amount_cents);
```

Index field	Role in this query
tenant_id	Equality search
created_seq	Requested order and output
amount_cents	Output without a table lookup

The plan reports USING COVERING INDEX; measured work is 129 instructions.

Column order determines the searchable prefix

```
SELECT sum(amount_cents)
FROM orders INDEXED BY idx
WHERE tenant_id = 42
  AND created_seq BETWEEN 80000 AND 119999
```

Index order	Searchable region	VM instructions
tenant_id, created_seq	One tenant within the sequence range	1,991
created_seq, tenant_id	Sequence range across tenants	200,808

These paths are selected with INDEXED BY to isolate key order in the experiment.

Coverage depends on the requested output columns

Selected values	Plan	Table records needed
created_seq, amount_cents	USING COVERING INDEX	No
created_seq, amount_cents, payload	USING INDEX	Yes

The index already holds

Tenant, sequence and amount are inside the composite key.

The extra value lives in the table

Adding payload requires row lookups.
This version executes
170 instructions.

A partial index represents a useful subset

```
CREATE INDEX idx ON orders(tenant_id, created_seq, amount_cents)
WHERE status = 'open';
```

Index	Rows represented	Index storage
Partial active index	20,045	0.336 MiB
Full: (status, tenant_id, created_seq, amount_cents)	200,000	4.641 MiB

The query includes status = 'open', so the partial index contains every eligible row.

An expression can change the usable search constraint

Predicate	Observed plan	VM instructions
tenant_id = 42	SEARCH ... (tenant_id=?)	5,953
tenant_id + 0 = 42	SCAN ... COVERING INDEX	801,994

Same answer, different access path

Both queries count the same 1,980 matching rows.

Make the intended access visible

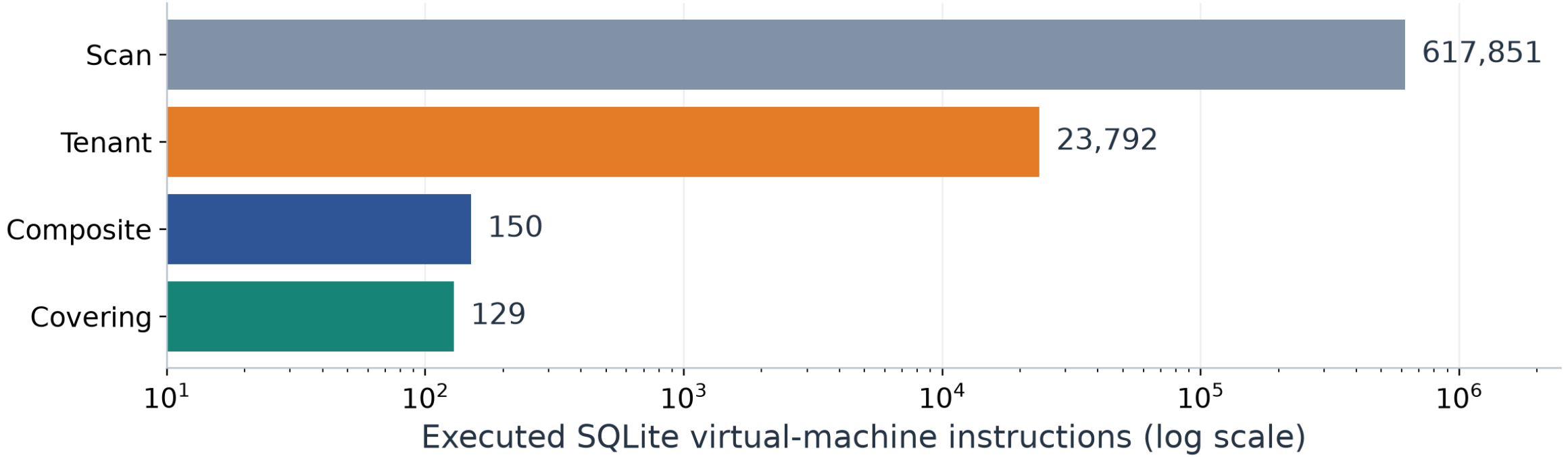
Keep the indexed equality directly searchable, or design an index for the actual expression being queried.

Read the query plan as a physical route

Plan detail	What to look for
SCAN	Traversal through the selected table or index
SEARCH ... (tenant_id=?)	A constrained lookup into a key range
USING COVERING INDEX	Requested fields supplied directly by the index
USE TEMP B-TREE FOR ORDER BY	A separate structure supplies sorting

Pair the plan with returned rows and measured work; each explains a different part of execution.

The same result can require radically different work



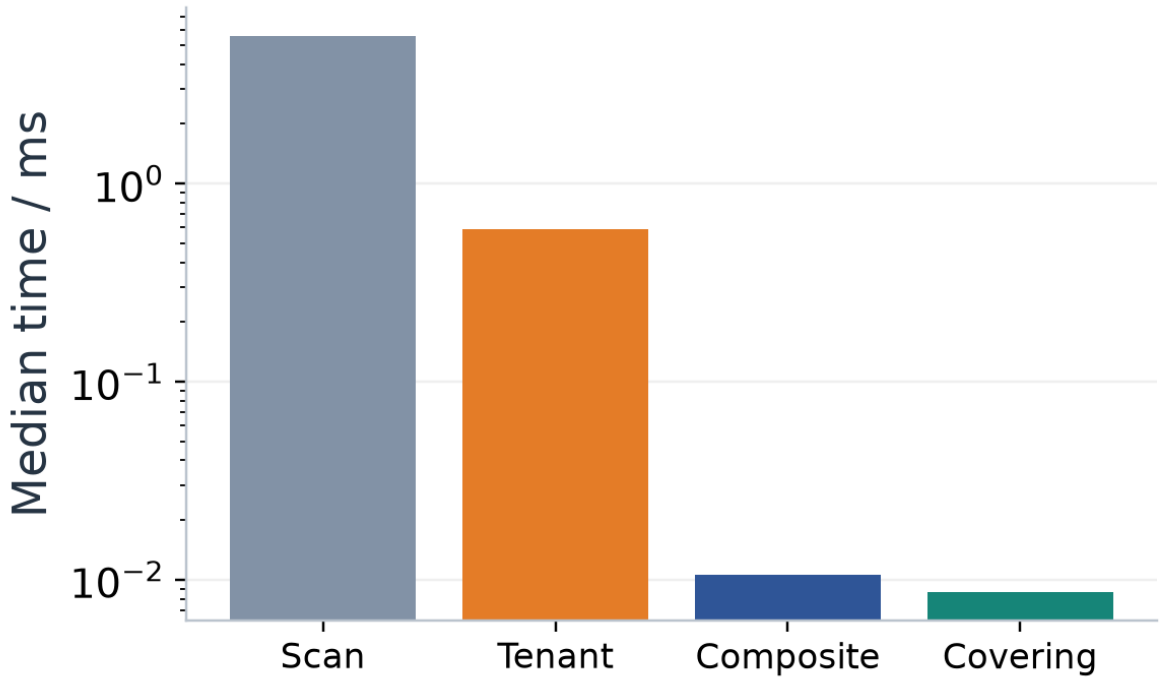
Progressively remove work

Search a tenant, preserve the order, then supply the output columns inside the index.

From the baseline to coverage

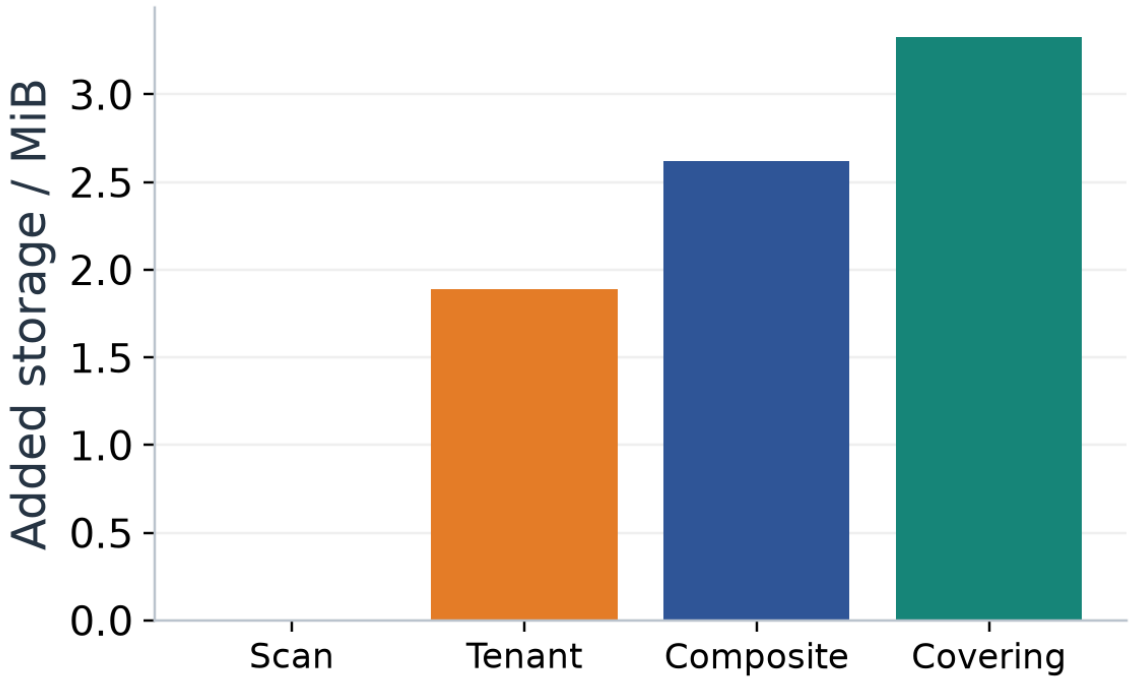
617,851 → 129 instructions.
All four cases return identical rows in identical order.

Latency and storage reveal different costs



Median warm-cache times

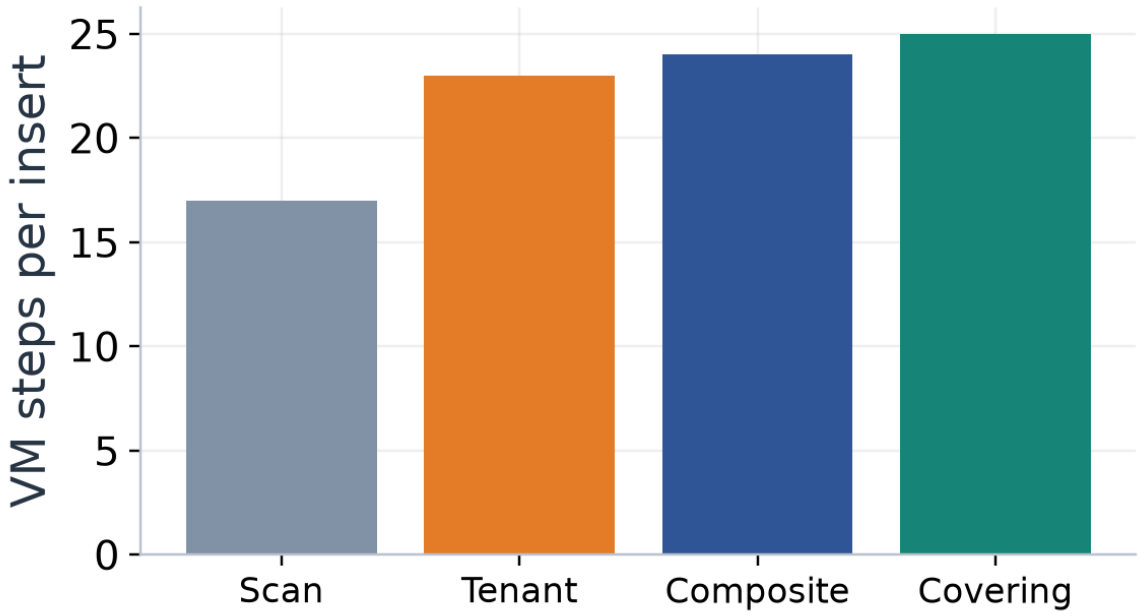
SQLite 3.53.1; in-memory database.
200,000 rows; 11 timed executions
after 3 warm-ups per query.

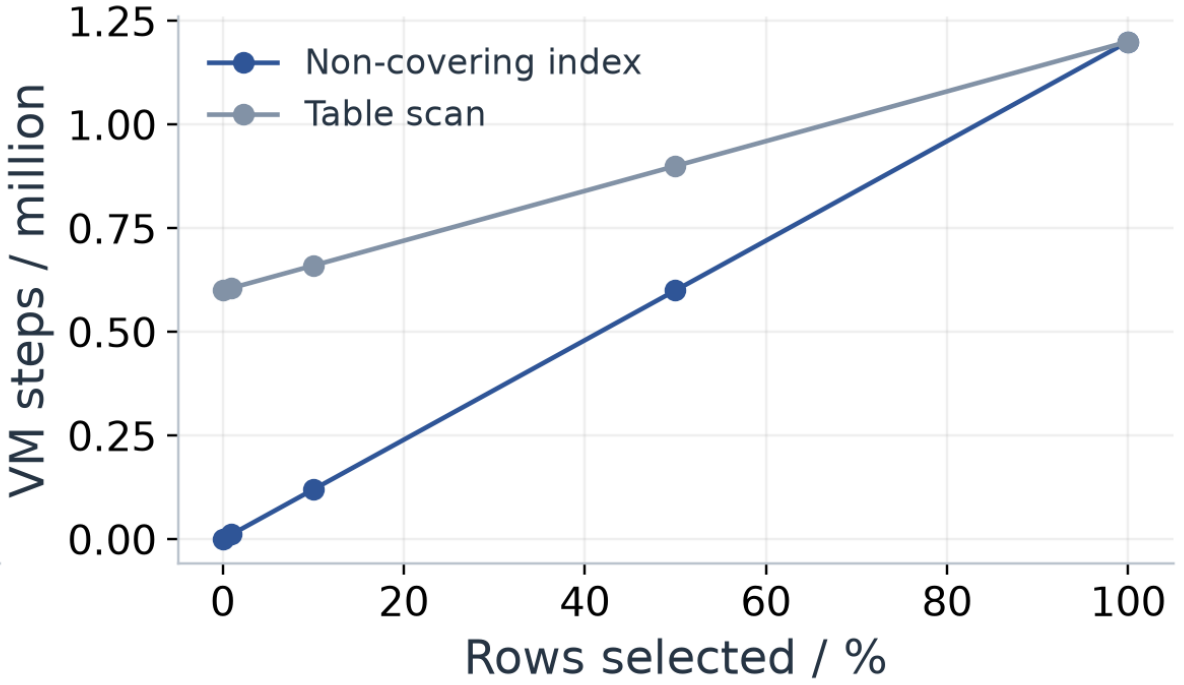


The added order takes space

Each index stores another key order.
The covering form also carries
amount_cents for direct output.

Index maintenance and broad reads also consume work





Each insert maintains every index

The left panel counts instructions for the same 5,000-row transaction, rolled back after measurement.

A broad query touches more records

The right panel forces each access path and fetches payload from the table. At 100%, both traverse the full dataset.

Build indexes through a repeatable query experiment

Step	Concrete action
1 Define the query	Fix predicates, ordering, output columns and row limit.
2 Capture the baseline	Save EXPLAIN QUERY PLAN, results and work measurements.
3 Choose the key	Place equality/range/order fields to match the access path.
4 Measure the trade-off	Check read work, storage and write maintenance.
5 Refresh statistics	Use ANALYZE or PRAGMA optimize, then inspect the plan again.

For each query, compare candidate indexes against that query’s fixed baseline result.

Choose the index that matches the work to be done

Query requirement	Index property to evaluate
Selective equality	A usable leading key
Ordered limited results	Order after equality-constrained key fields
Output without row lookups	Coverage of every selected value
Active subset	A partial-index predicate implied by the query
Frequent writes or broad reads	Maintenance cost, storage and actual access work

An index helps by narrowing, ordering or supplying data along the query's actual path.

SQLite documentation: Query Planning · EXPLAIN QUERY PLAN · Partial Indexes